تمرين للتحضير لـ PLF: تأجير المركبات

> الصف: 5AKIF
> تدريب على PLF (الاختبار العملي) يوم الأربعاء، 4 نوفمبر 2026
> مدة العمل في PLF: 3 UE (حصص دراسية)

ملاحظات حول هذا التمرين

يوضح لك هذا التمرين شكل نص المهمة وبرنامج الاختبار في PLF. لـ PLF نفس البنية: نموذج للفئات (classes)، ووصف للفئات، وبرنامج اختبار يمنح النقاط.

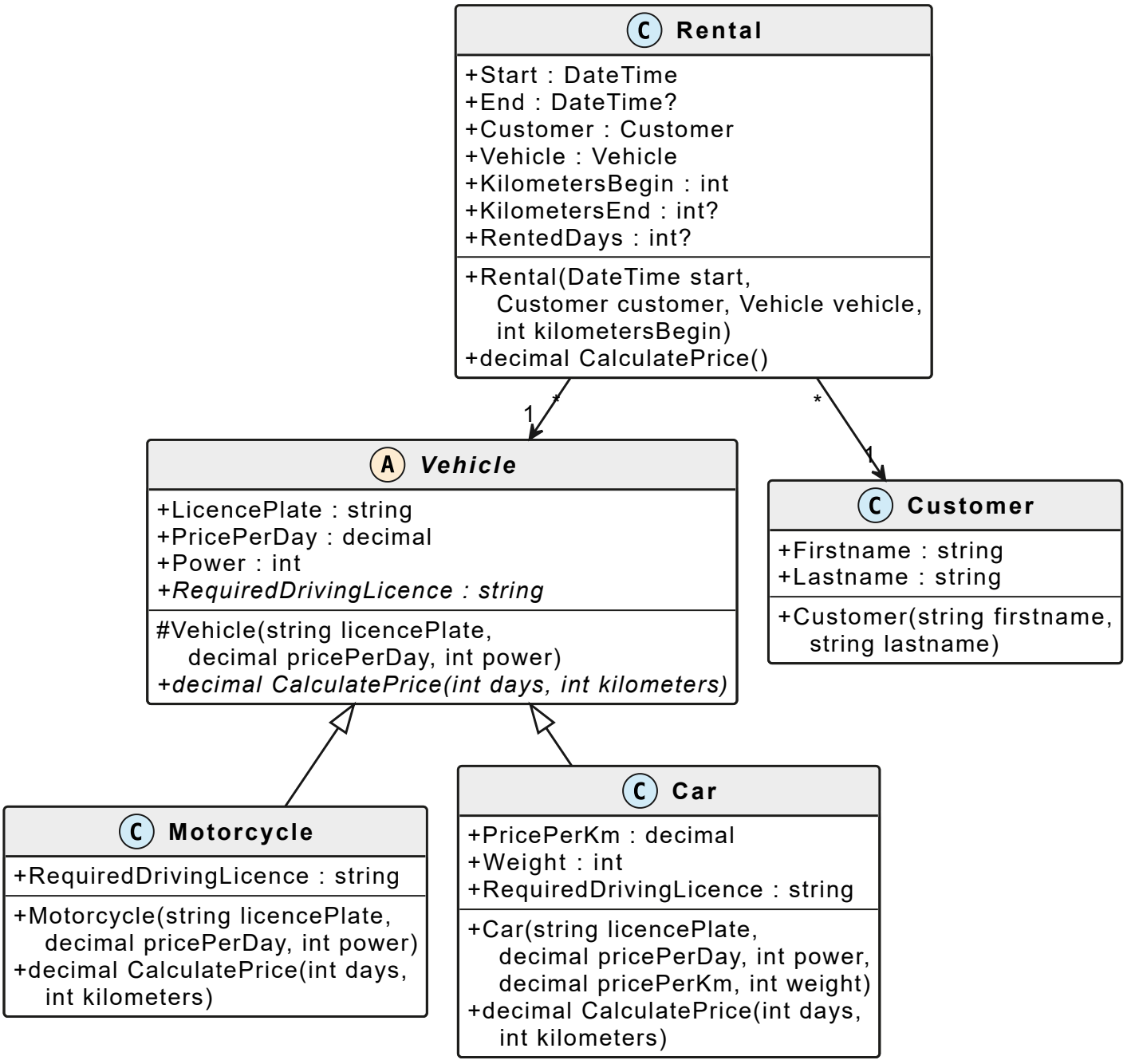
نزّل الملف *CarRentalService.7z* من تمارين الفصل *Types*. فك ضغط الملف وافتح الملف *CarRentalService.sln*.

> **ملاحظة**
> في PLF ستجد الـ solution على القرص *Z*. هناك اكتب أولًا اسمك في الملف *README.md*. أما في هذا التمرين فالملف *README.md* هو نص المهمة، ولا تحتاج إلى كتابة أي شيء فيه.

المهمة: تأجير المركبات

تؤجّر شركة تأجير دراجات نارية وسيارات. عند استئجار دراجة نارية تدفع سعرًا عن كل يوم. عند استئجار سيارة تدفع بالإضافة إلى ذلك عن كل كيلومتر تقطعه. لكل عملية تأجير يحفظ النظام سجل تأجير (*Rental*). تبقى عملية التأجير مفتوحة حتى تُعاد المركبة.

نموذج الفئات



وصف الفئات

أنشئ بنفسك الحقول الخاصة (private fields) التي تحتاجها في التنفيذ.

الفئة Vehicle

الفئة الأساسية المجردة (abstract base class) للسيارات والدراجات النارية. جميع الخصائص (properties) للقراءة فقط (read-only).

- **LicencePlate** هي لوحة أرقام المركبة.
- **PricePerDay** هو سعر الإيجار عن كل يوم بدأ.
- **Power** هي قدرة المركبة بالكيلوواط (kW).
- المُنشئ (constructor) من نوع *protected*.
- **RequiredDrivingLicence** خاصية مجردة (abstract property). تُرجع فئة رخصة القيادة اللازمة لهذه المركبة.
- **CalculatePrice()** دالة مجردة (abstract method). تحسب السعر من عدد أيام الإيجار (*days*) والكيلومترات المقطوعة (*kilometers*).

الفئة Motorcycle

جميع الخصائص للقراءة فقط.

- **RequiredDrivingLicence** تُرجع دائمًا "*A*".
- **()CalculatePrice** تحسب السعر بالصيغة *PricePerDay* × *days*. لا تؤثر الكيلومترات على السعر.

الفئة Car

جميع الخصائص للقراءة فقط.

- **PricePerKm** هو السعر عن كل كيلومتر مقطوع.
- **Weight** هو الوزن بالكيلوغرام.
- **RequiredDrivingLicence** تُرجع "*B*" إذا كان وزن السيارة أقل من 3500 كغ. وإلا فإنها تُرجع "*C*" (شاحنة).
- **()CalculatePrice** تحسب السعر بالصيغة *PricePerKm* × *kilometers* + *PricePerDay* × *days*.

الفئة Customer

تحفظ الفئة الاسم الأول واسم العائلة للزبون فقط. يضبط المُنشئ قيم جميع الخصائص. جميع الخصائص للقراءة فقط.

الفئة Rental

تحفظ الفئة عملية تأجير واحدة.

- **Start** و**Customer** و**Vehicle** و**KilometersBegin** تُضبط في المُنشئ، وهي للقراءة فقط.
- **Start** هي بداية الإيجار.
- **KilometersBegin** هي قراءة عدّاد الكيلومترات في بداية الإيجار.
- **End** هي نهاية الإيجار. قيمتها *null* ما دامت المركبة لم تُعَد بعد. عند إعادة المركبة تُضبط قيمتها من خارج الفئة.
- **KilometersEnd** هي قراءة عدّاد الكيلومترات في نهاية الإيجار. قيمتها *null* ما دامت المركبة لم تُعَد بعد. عند إعادة المركبة تُضبط قيمتها من خارج الفئة.
- **RentedDays** هي عدد الأيام التي بدأت، وهي للقراءة فقط. إذا كانت *End* تساوي *null* فإن *RentedDays* تساوي *null* أيضًا. الإيجار الذي ينتهي في اليوم نفسه مدته يوم واحد.
- **()CalculatePrice** تحسب سعر الإيجار:
  - إذا كانت *End* تساوي *null* تُرجع الدالة 0.
  - إذا كانت *KilometersEnd* تساوي *null* تُرجع الدالة 0.
  - وإلا فإنها تُرجع نتيجة *()CalculatePrice* للمركبة المستأجرة. مرّر لها *RentedDays* والكيلومترات المقطوعة (*KilometersEnd* − *KilometersBegin*).

مثال: تُستأجر سيارة قيمة *PricePerDay* فيها 10 يورو وقيمة *PricePerKm* فيها 5 يورو من 10.11.2021 إلى 15.11.2021. قراءة العدّاد 100 كم في البداية و200 كم في النهاية.

- *RentedDays* تساوي 6 (10 و11 و12 و13 و14 و15 نوفمبر).
- *()CalculatePrice* تُرجع 560 = 5 × 100 + 10 × 6 يورو.

نصيحة

- عندما تطرح قيمتين من نوع *DateTime* تحصل على *TimeSpan*. تحصل على عدد الأيام التي بدأت بهذا التعبير:

```csharp
(int)(end - start).TotalDays + 1
```

- هذا يعمل فقط مع *DateTime*، وليس مع *DateTime?*. لذلك تحقق أولًا مما إذا كانت *End* تساوي *null*. بعد ذلك تحصل على القيمة عن طريق `End.Value`. وينطبق الشيء نفسه على *KilometersEnd*.
- تكتب قيمة حرفية (literal) من نوع *decimal* مع *M* في النهاية، مثلًا `0.5M`.

المطلوب

أنشئ كل فئة في ملف مستقل. **لا** يجوز لك تعديل الملفين *Program.cs* و*TestHelpers.cs*.

Program.cs هو برنامج اختبار. لا يُترجَم (compile) إلا إذا وُجدت جميع الفئات مع جميع الخصائص والدوال الموجودة في النموذج. انتبه أيضًا إلى أسماء معاملات المُنشئ، لأن برنامج الاختبار يستخدمها (مثلًا `pricePerDay: 10`). لذلك أنشئ جميع الفئات أولًا. الدوال والخصائص التي لم تنفذها بعد ترمي *NotImplementedException*. بعد ذلك شغّل البرنامج في المشروع *CarRentalService.Application*. يعرض البرنامج لكل فحص *OK* أو *FAILED*. إذا رمى فحصٌ استثناءً (exception) فإنه يُعدّ غير ناجح. الاستثناء خارج الفحص (مثلًا عند إنشاء بيانات الاختبار) يُنهي البرنامج.

الحل الكامل يعرض *OK* في جميع الفحوص الـ 43. في النهاية يعرض البرنامج النقاط التي حصلت عليها والعلامة:

```text
Result of the program in C:\...\CarRentalService.Application:
43 of 43 checks passed.
48 of 48 points. Grade: Sehr gut.
```

التقييم

للحصول على تقييم إيجابي يجب أن يُترجَم البرنامج بدون أخطاء. البرامج التي لا تُترجَم أو تتوقف بسبب خطأ تحصل على العلامة *Nicht genügend* (راسب). يعرض الجدول جميع فحوص برنامج الاختبار. النصوص هي نفس النصوص التي تظهر على الشاشة. كل فحص نتيجته *OK* يمنح النقاط الموجودة في العمود *P*.

رقم	الفحص (48 نقطة في المجموع)	P
Checking class implementation		
1	Rental does not have a default constructor.	1
2	Rental.Start is read-only.	1
3	Rental.Customer is read-only.	1
4	Rental.Vehicle is read-only.	1
5	Rental.KilometersBegin is read-only.	1
6	Rental.RentedDays is read-only.	1
7	Rental.End, KilometersEnd and RentedDays are nullable.	1
8	Customer does not have a default constructor.	1
9	Customer is immutable.	1

رقم	الفحص (48 نقطة في المجموع)	P
10	Vehicle does not have a default constructor.	1
11	Vehicle is abstract.	1
12	Vehicle has a protected constructor.	1
13	Vehicle is immutable.	1
14	Vehicle.PricePerDay is decimal and Vehicle.Power is int.	1
15	Vehicle.RequiredDrivingLicence is abstract.	1
16	Vehicle.CalculatePrice() is abstract.	1
17	Motorcycle does not have a default constructor.	1
18	Motorcycle is immutable.	1
19	Car does not have a default constructor.	1
20	Car is immutable.	1
21	Car.PricePerKm is decimal and Car.Weight is int.	1
Checking constructors		
22	Constructor of Customer sets the values.	1
23	Constructor of Motorcycle sets the values.	1
24	Constructor of Car sets the values.	1
25	Constructor of Rental sets the values.	1
Checking Motorcycle		
26	The required driving licence for motorcycles is A.	1
27	Motorcycle.CalculatePrice() returns days x PricePerDay.	2
Checking Car		
28	The required driving licence for cars < 3500 kg is B.	1
29	Car.CalculatePrice() returns days x PricePerDay + km x PricePerKm.	2
30	The required driving licence for cars >= 3500 kg is C.	1
Checking Rental		
31	The default value of Rental.KilometersEnd is null.	1
32	The default value of Rental.End is null.	1

رقم	الفحص (48 نقطة في المجموع)	P
33	Without an end date Rental.RentedDays is null.	1
34	Without End and KilometersEnd Rental.CalculatePrice() returns 0.	1
35	Rental.RentedDays returns the number of days started.	2
36	Without KilometersEnd Rental.CalculatePrice() returns 0.	1
37	Rental.CalculatePrice() uses the driven kilometers.	2
38	Without End Rental.CalculatePrice() returns 0.	1
39	Rental.RentedDays is 1 if the vehicle is returned on the same day.	1
40	Rental.RentedDays works across the end of a month.	1
41	Rental.RentedDays works across the end of a year.	1
42	Rental.CalculatePrice() charges the days if no kilometers were driven.	1
43	Rental.CalculatePrice() uses the price calculation of the rented vehicle.	2

من 42 إلى 48 نقطة	*Sehr gut* (1، ممتاز)
من 36 إلى 41 نقطة	*Gut* (2، جيد)
من 30 إلى 35 نقطة	*Befriedigend* (3، مقبول)
من 24 إلى 29 نقطة	*Genügend* (4، كافٍ)
من 0 إلى 23 نقطة	*Nicht genügend* (5، راسب)

يعرض البرنامج اسم العلامة باللغة الألمانية.